COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010/Data Structures and Algorithms	**SECTION:**	CPE21S3
DATE PERFORMED:	September 8, 2026	**DATE SUBMITTED:**	September 8, 2026
NAME:	Jaime D.C. Godino Jr.	**INSTRUCTOR:**	Engr. Jimlord Quejado

ACTIVITY NO. 7.1 **Sorting Algorithms**

1. PROCEDURE OUTPUT

ILO A: Create C++ code for implementation of selection, bubble, insertion and merge sort.

Code + Console Screenshot

sortingAlgo.h

```
4   // Generate an array containing 100 random values
5   template <typename T>
6   void generateArray(T arr[], int size) {
7       for (int i = 0; i < size; i++) {
8           arr[i] = rand() % 100 + 1;
9       }
10  }
11
12  // Display all elements inside the array
13  template <typename T>
14  void displayArray(T arr[], int size) {
15      for (int i = 0; i < size; i++) {
16          std::cout << arr[i] << " ";
17
18          if ((i + 1) % 10 == 0) {
19              std::cout << std::endl;
20          }
21      }
22  }
```

main.cpp

```cpp
#include <iostream>
#include <cstdlib>
#include <time.h>
#include "sortingAlgo.h"

int main() {
    const int maxSize = 100;
    int arr[maxSize];

    // Seed for random values
    srand(time(0));

    // Generate 100 random values
    generateArray(arr, maxSize);

    std::cout << "Unsorted Array:" << std::endl;
    displayArray(arr, maxSize);
```

output

```
Unsorted Array:
84 91 1 20 95 36 81 35 34 11
56 11 84 68 26 17 93 21 33 95
69 85 22 12 35 45 32 14 76 100
33 7 7 92 97 43 11 34 18 28
99 29 84 63 41 80 100 54 20 22
79 83 70 50 18 16 38 9 56 63
19 100 32 40 34 28 47 64 92 99
95 45 74 80 61 100 80 22 49 6
20 83 80 69 53 99 86 76 64 62
100 78 53 66 8 8 21 91 60 26
```

Observations	These lines of code generate 100 random integers between 1 and 100 using time.h. The same unsorted array should be reused across all algorithms for the same comparisons. This makes the runtime and comparison counts fast for the algorithms.

Table 7-1. Array of Values for Sort Algorithm Testing

Code + Console Screenshot	**sortingAlgo.h**
	`template <typename T>`
	`void bubbleSort(T arr[], int maxSize) {`
	`  for (int i = 0; i < maxSize; i++) {`
	`  for (int j = i + 1; j < maxSize; j++) {`
	`  if (arr[i] > arr[j]) {`
	`  std::swap(arr[i], arr[j]);`
	`  }`
	`  }`
	`  }`
	`}`
	
	main.cpp
	`std::cout << "\nBubble Sort:" << std::endl;`
	`bubbleSort(arr, maxSize);`
	`displayArray(arr, maxSize);`
	
	output
	`Bubble Sort:`
	`1 6 7 7 8 8 9 11 11 11`
	`12 14 16 17 18 18 19 20 20 20`
	`21 21 22 22 22 26 26 28 28 29`
	`32 32 33 33 34 34 34 35 35 36`
	`38 40 41 43 45 45 47 49 50 53`
	`53 54 56 56 60 61 62 63 63 64`
	`64 66 68 69 69 70 74 76 76 78`
	`79 80 80 80 80 81 83 83 84 84`
	`84 85 86 91 91 92 92 93 95 95`
	`95 97 99 99 99 100 100 100 100 100`

Observations	This implementation compares arr[i] against every element after it and swaps whenever a smaller element is found further down, basically selecting the minimum through repeated swapping. It performs a full pass for each of the n elements, with an inner loop scanning all remaining elements each time. The time complexity is $O(n^2)$ in the best, average, and worst cases.

Table 7-2. Bubble Sort Technique

Code + Console Screenshot

sortingAlgo.h

```
35  template <typename T>
36  int routine_smallest(T arr[], int k, int arrSize) {
37      int position = k;
38
39      T smallestElement = arr[k];
40
41      for (int j = k + 1; j < arrSize; j++) {
42          if (arr[j] < smallestElement) {
43              smallestElement = arr[j];
44              position = j;
45          }
46      }
47
48      return position;
49  }
50
51  template <typename T>
52  void selectionSort(T arr[], int arrSize) {
53      int min;
54
55      for (int i = 0; i < arrSize - 1; i++) {
56          min = routine_smallest(arr, i, arrSize);
57          std::swap(arr[i], arr[min]);
58      }
59  }
```

main.cpp

```
23      std::cout << "\nSelection Sort:" << std::endl;
24      selectionSort(arr, maxSize);
25      displayArray(arr, maxSize);
```

output

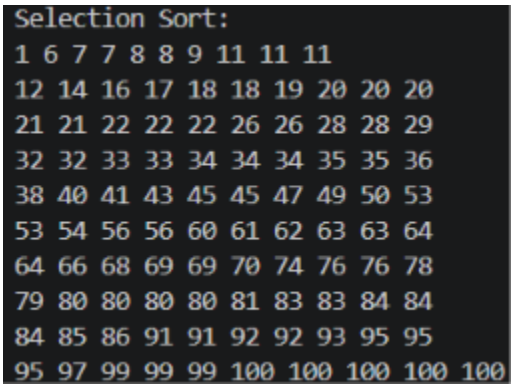
	
Observations	This implementation works by scanning the unsorted portion of the array to find the location of the minimum element using a helper function, then swapping it into its correct position at index i. The outer loop runs n - 1 times, executing n -1 different passes to place all elements in order. Regardless of initial array order, the inner loop always scans the remaining unsorted elements, which results in a time complexity of $O(n^2)$ across best, average, and worst cases.

Table 7-3. Selection Sort Algorithm

Code + Console Screenshot	**sortingAlgo.h**
	`template <typename T>`
	`void insertionSort(T arr[], int arrSize) {`
	`  int k = 1, j;`
	`  T temp;`
	
	`  while(k < arrSize) {`
	`  temp = arr[k];`
	`  j = k - 1;`
	`  while( j >= 0 && temp <= arr[j]){`
	`  arr[j + 1] = arr[j];`
	`  j--;`
	`  }`
	`  arr[j+1] = temp;`
	`  k++;`
	`  }`
	`}`
	`#endif`
	main.cpp
	`std::cout << "\nInsertion Sort:" << std::endl;`
	`insertionSort(arr, maxSize);`
	`displayArray(arr, maxSize);`
	output

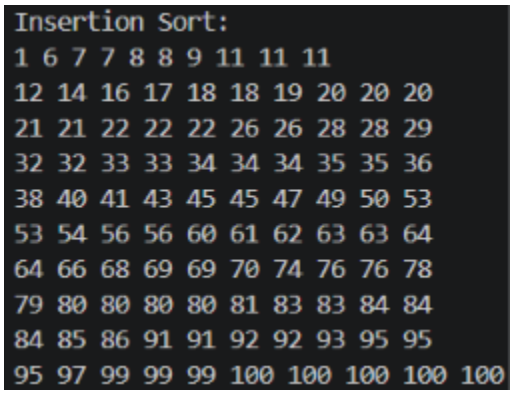
	
Observations	This implementation builds the final sorted array one element at a time by picking the current element temp at index k and changing larger preceding elements one position to the right, The main while loop runs from k = 1 to n - 1, performing n - 1 total passes over the array, While its worst and average-case time complexities $O(n^2)$ due to nested shifting, it can achieve a best-case time complexity of O(n) when the input array is already sorted.

Table 7-4. Insertion Sort Algorithm

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

ILO B: Solve given data sorting problems using appropriate basic sorting algorithms

Candidate 1	Bo Dalton Capistrano
Candidate 2	Cornelius Raymon Agustín
Candidate 3	Deja Jayla Bañaga
Candidate 4	Lalla Brielle Yabut
Candidate 5	Franklin Relano Castro

Problem: Generate an array A[0…100] of unsorted elements, wherein the values in the array are indicative of a vote to a candidate. This means that the values in your array must only range from 1 to 5. Using sorting and searching techniques, develop an algorithm that will count the votes and indicate the winning candidate.

NOTE: The sorting techniques you have the option of using in this activity can be either bubble, selection, or insertion sort. Justify why you chose to use this sorting algorithm.

Deliverables:

- Pseudocode of Algorithm
- Screenshot of Algorithm Code
- Output Testing

Pseudocode of the Algorithm

```
START

        Set SIZE = 101

        Create array A[0...100]

        FOR i = 0 TO SIZE - 1
                A[i] = random number from 1 to 5
        END FOR

        // Insertion Sort
        FOR i = 1 TO SIZE - 1
                key = A[i]
                j = i - 1

                WHILE j >= 0 AND A[j] > key
                A[j + 1] = A[j]
                j = j - 1
                END WHILE

                A[j + 1] = key
        END FOR

        Set count1 = 0
        Set count2 = 0
        Set count3 = 0
        Set count4 = 0
        Set count5 = 0

        FOR i = 0 TO SIZE - 1
                IF A[i] == 1
                count1 = count1 + 1
                ELSE IF A[i] == 2
                count2 = count2 + 1
                ELSE IF A[i] == 3
                count3 = count3 + 1
                ELSE IF A[i] == 4
                count4 = count4 + 1
                ELSE IF A[i] == 5
                count5 = count5 + 1
                END IF
        END FOR

        DISPLAY count1 as votes for Bo Dalton Capistrano
        DISPLAY count2 as votes for Cornelius Raymon Agustín
        DISPLAY count3 as votes for Deja Jayla Bañaga
```

```
        DISPLAY count4 as votes for Lalla Brielle Yabut
        DISPLAY count5 as votes for Franklin Relano Castro

        //Find the highest vote count

        IF count1 is highest
                Winner = Bo Dalton Capistrano
        ELSE IF count2 is highest
                Winner = Cornelius Raymon Agustín
        ELSE IF count3 is highest
                Winner = Deja Jayla Bañaga
        ELSE IF count4 is highest
                Winner = Lalla Brielle Yabut
        ELSE
                Winner = Franklin Relano Castro
        END IF

        Display Winner

END
```

Screenshot of Algorithm Code

supplementalSortingAlgo.h

```
#ifndef SORTING_H
#define SORTING_H

//generate an array of random votes from 1 to 5
template <typename T>
void generateVotes(T arr[], int size) {
    for (int i = 0; i < size; i++) {
        arr[i] = rand() % 5 + 1;
    }
}

//display the array
template <typename T>
void displayArray(T arr[], int size) {
    for (int i = 0; i < size; i++) {
        std::cout << arr[i] << " ";

        if ((i + 1) % 10 == 0) {
            std::cout << std::endl;
        }
    }

    if (size % 10 != 0) {
        std::cout << std::endl;
    }
}

```

```cpp
// Insertion Sort
template <typename T>
void insertionSort(T arr[], int size) {
    for (int i = 1; i < size; i++) {
        T key = arr[i];
        int j = i - 1;

        while (j >= 0 && arr[j] > key) {
            arr[j + 1] = arr[j];
            j--;
        }

        arr[j + 1] = key;
    }
}

//count the votes
template <typename T>
void countVotes(T arr[], int size, int voteCount[]) {
    for (int i = 0; i < size; i++) {
        if (arr[i] >= 1 && arr[i] <= 5) {
            voteCount[arr[i]]++;
        }
    }
}

```

```cpp
//display voting results and determine winner
void displayResults(int voteCount[]) {

    std::string candidates[6] = {
        "",
        "Bo Dalton Capistrano",
        "Cornelius Raymon Agustín",
        "Deja Jayla Bañaga",
        "Lalla Brielle Yabut",
        "Franklin Relano Castro"
    };

    std::cout << "\n===== VOTING RESULTS =====\n";

    for (int i = 1; i <= 5; i++) {
        std::cout << "Candidate " << i << " - "
                  << candidates[i] << ": "
                  << voteCount[i] << " votes"
                  << std::endl;
    }

    int winner = 1;

    for (int i = 2; i <= 5; i++) {
        if (voteCount[i] > voteCount[winner]) {
            winner = i;
        }
    }

```

```
83      std::cout << "\n===== WINNER =====\n";
84      std::cout << candidates[winner]
85                << " with " << voteCount[winner]
86                << " votes!"
87                << std::endl;
88  }
89
90  #endif
```

supplementalMain.cpp

```cpp
#include <iostream>
#include <cstdlib>
#include <time.h>
#include "supplementalSortingAlgo.h"

using namespace std;

int main() {

    const int SIZE = 101;

    //array for the votes
    int A[SIZE];

    //array for counting votes
    //index 1 to 5 represents the five candidates
    int voteCount[6] = {0};

    srand(time(0));

    //generate random votes from 1 to 5
    generateVotes(A, SIZE);

    cout << "===== UNSORTED VOTES =====\n";
    displayArray(A, SIZE);

    //sort the votes using Insertion Sort
    insertionSort(A, SIZE);

    cout << "\n===== SORTED VOTES =====\n";
    displayArray(A, SIZE);
```

```
32
33      //count the votes
34      countVotes(A, SIZE, voteCount);
35
36      //display the results and winner
37      displayResults(voteCount);
38
39      return 0;
40  }
```

output

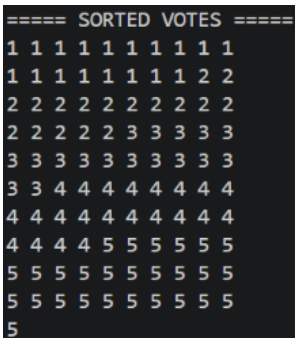
```
===== UNSORTED VOTES =====
2 3 3 4 1 5 5 5 2 1
2 5 1 2 1 4 4 4 3 1
5 5 2 2 1 4 1 5 4 2
1 4 4 3 3 4 5 5 2 4
4 5 4 1 1 2 1 1 4 2
5 3 2 5 4 1 5 3 4 5
3 5 2 1 5 1 4 1 2 5
4 5 3 2 4 5 4 3 5 4
1 3 2 2 5 3 1 3 3 5
3 5 5 2 5 3 5 5 4 4
3
```

```
===== SORTED VOTES =====
1 1 1 1 1 1 1 1 1 1
1 1 1 1 1 1 1 1 2 2
2 2 2 2 2 2 2 2 2 2
2 2 2 2 2 3 3 3 3 3
3 3 3 3 3 3 3 3 3 3
3 3 4 4 4 4 4 4 4 4
4 4 4 4 4 4 4 4 4 4
4 4 4 4 5 5 5 5 5 5
5 5 5 5 5 5 5 5 5 5
5 5 5 5 5 5 5 5 5 5
5
```

```
===== VOTING RESULTS =====
Candidate 1 - Bo Dalton Capistrano: 18 votes
Candidate 2 - Cornelius Raymon Agust├¡n: 17 votes
Candidate 3 - Deja Jayla Ba├▒aga: 17 votes
Candidate 4 - Lalla Brielle Yabut: 22 votes
Candidate 5 - Franklin Relano Castro: 27 votes

===== WINNER =====
Franklin Relano Castro with 27 votes!
```

Output Testing

Output Console Showing Sorted	Manual Count	Array Count Result of Algorithm
	18	Candidate 1 - Bo Dalton Capistrano: 18 votes
	17	Candidate 2 - Cornelius Raymon Agustin: 17 votes
	17	Candidate 3 - Deja Jayla Baraga: 17 votes
	22	Candidate 4 - Lalla Brielle Yabut: 22 votes
	27	Candidate 5 - Franklin Relano Castro: 27 votes

Question: Was your developed vote counting algorithm effective? Why or why not?
Yes, the program is effective because it was able to sort the 100 botes, count the number of votes received by each candidate, and determine the candidate with the highest number of votes. Using insertion sort made the values easier to group and count because the same vote numbers were placed together. Finally, the program was able to print the vote totals and the winner, which shows that it worked as intended.

Conclusion:

Through this activity, I learned how bubble, selection, and insertion sort each approach the same goal of ordering data differently. In the procedure, generating a reusable 100-element array and running it through all three algorithms made it easy to observe these differences directly in the console output. For the supplementary activity, applying insertion sort to a 101-element array of vote values from 1 to 5 and then tallying the results using indexed counters, followed by manually cross-checking the sorted output, confirmed that the program correctly grouped identical vote values together and produced accurate candidate totals. Overall, I think I performed well in this activity since I was able to implement all the required sorting techniques correctly, but I recognize that my main area for improvement is deepening my understanding of when to apply searching techniques like binary search more efficiently.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **Grammarly and ChatGPT**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Grammar correction and explanation of concepts**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **I used Grammarly to fix and improve my grammar while using ChatGPT to further understand the concept while working on the programs in the activity**

COMPUTER ENGINEERING

QUEZON CITY

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.